

Project 1 — readme

Name: Israk Farhan Student number: 301412948

1. AI use (~1 page)

AI assistance is allowed. The condition is that you understand every line you submit and can explain it if asked.

If you used no AI tools at all, delete the questions below and write this one line:

I did not use AI tools on this project.

Otherwise, answer all of them.

Which two or three prompts did the most work?

I used ChatGPT occasionally at different points in the project when I needed extra guidance. The most helpful prompts were about understanding HTTP request and response handling, working with concurrency and the thread pool, and getting guidance for the two bonus sections.

What did the AI get wrong, leave incomplete, or state confidently but incorrectly?

A few of the suggestions needed to be corrected. For example, one test tried to use `socket.socketpair()` directly as a context manager, which caused a `TypeError`. Another early thread-pool version did not handle unexpected client disconnects, so a worker thread could stop running.

What did you do instead?

I fixed the socket test and added error handling so worker threads would continue running after a client disconnected unexpectedly. I also tested the code myself in VS Code through Ubuntu WSL, compared binary files byte-for-byte, checked the HTTP responses, and ran concurrency tests. When a test exposed a problem, I corrected it and tested again before moving on.

2. Design (~1 page)

A short paragraph for each question, about one page in total. Write about your code, not about HTTP in general.

How does your server know a request has ended?

My server determines that a request header is complete by using `recv_request_head()`. Because TCP provides a stream of bytes, the whole request is not guaranteed to arrive in one `recv()` call. The function therefore keeps collecting bytes until it finds `\r\n\r\n`, which marks the end of the HTTP header section. The completed header is then passed to `handle_request()`. If the client closes the connection before a full request arrives, the function returns `None` and the server closes that connection.

Why does your client not simply read until the connection closes?

The client does not wait for the server to close the socket because HTTP/1.1 connections can remain open for more requests. Instead, it reads the `Content-Length` header and uses that value to know exactly how many body bytes belong to the current response. `read_head()` keeps any body bytes that arrived with the headers, and

`read_body()` finishes reading the required amount while preserving extra bytes for the next response. This is what allows several requests to use the same TCP connection without waiting for a timeout.

How does your thread pool bound concurrency — and what did you observe when you tested it?

The server limits concurrency by creating a fixed number of worker threads from the `--workers` value. Accepted connections are placed in a shared queue, and each worker handles one connection at a time, so extra clients must wait until a worker is available. A `threading.Lock` protects the shared request counter while it is updated. With two workers and six clients, I observed that two were handled while the other four waited. I also ran 320 concurrent requests successfully and confirmed that the counter contained every value from 1 through 320 with no duplicates or gaps.

How this document is marked (not counted in the page limit)

5 marks for each section, awarded 0, 3 or 5.

	0	3	5
AI use	absent	present but generic — could describe any project	specific: real prompts, a real mistake, what you did about it
Design	absent, or plainly not about your own code	broadly right, thin	correct and specific about your own code, not about HTTP in general

This document is worth 10 of the project's 100 points. If it is missing from your submission, it scores 0 on this item and affects nothing else — the automated marks for the five tasks are unaffected.

You may also be picked at random for a short oral check-in and asked to walk through your own code. The design questions above are the questions you will be asked, so answering them honestly here is the preparation.